Remote Code Execution via Unsafe Deserialization in AWS Lambda

Course: ICS-344 Information Security · KFUPM · DVSA Project
OWASP Category: A08:2021 – Software and Data Integrity Failures
Affected Function: `DVSA-ORDER-MANAGER`
Severity: Critical
Status: ✅ Fixed and Verified

1. Goal and Vulnerability Summary

Lesson 1 demonstrates an Event Injection vulnerability in the `DVSA-ORDER-MANAGER` Lambda function. The affected components are AWS API Gateway and Lambda. The function uses the insecure `node-serialize` library to deserialize user-controlled input, allowing an attacker to inject and execute arbitrary JavaScript code on the server.

Security Impact: Full Remote Code Execution (RCE) on the Lambda backend — without authentication.

2. Why This Works / Root Cause

The `node-serialize` library accepts serialized JavaScript functions marked with `_$$ND_FUNC$$_`. When the attacker wraps malicious code in this marker, the library executes it during deserialization — before any validation occurs. This means any user sending a crafted request can run arbitrary code on the server without authentication.

Unlike `JSON.parse()`, which only handles plain data, `node-serialize` was designed to reconstruct JavaScript objects including live functions. This makes it inherently dangerous when used on attacker-controlled input.

Root cause: Passing untrusted user input directly to `unserialize()` from the `node-serialize` library.

3. Environment and Setup

Component	Detail
Platform	AWS Lambda (Node.js)
API Endpoint	`https://qo2xzjt4dc.execute-api.us-east-1.amazonaws.com/Stage/order`
Vulnerable Function	`DVSA-ORDER-MANAGER`
CloudWatch Log Group	`/aws/lambda/DVSA-ORDER-MANAGER`
Vulnerable Library	`node-serialize` (CVE-2017-5941)
Tools Used	`curl` (Mac Terminal), AWS CloudWatch Console

4. Reproduction Steps

Step 1. Open Terminal on Mac.

Step 2. Run the following injection payload — no authorization header is required:

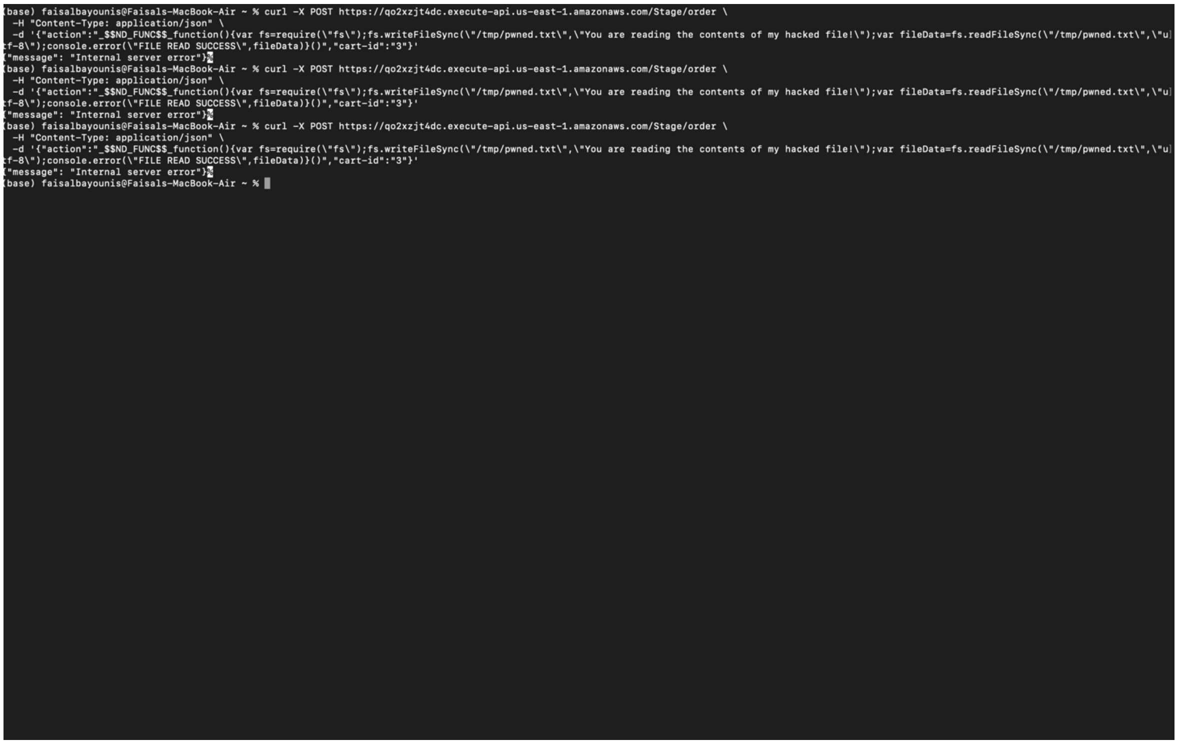
```
curl -X POST https://qo2xzjt4dc.execute-api.us-east-1.amazonaws.com/Stage/order \
  -H "Content-Type: application/json" \
  -d '{"action":"_$$ND_FUNC$$_function(){var
fs=require(\"fs\");fs.writeFileSync(\"/tmp/pwned.txt\",\"You are reading the contents
of my hacked file!\");var fileData=fs.readFileSync(\"/tmp/pwned.txt\",\"utf-
8\");console.error(\"FILE READ SUCCESS\",fileData)}()","cart-id":"3"}'
```

Step 3. The terminal returns `{"message": "Internal server error"}` — this is expected and does **NOT** mean the attack failed. API Gateway hides the backend crash and returns a generic 500-style error to the client.

Step 4. Go to AWS Console → **CloudWatch** → **Log groups** → `/aws/lambda/DVSA-ORDER-MANAGER`.

Step 5. Open the most recent log stream and search for `FILE READ SUCCESS`.

[INSERT SCREENSHOT — Terminal showing the curl command and Internal server error response]



> **Note:** The important evidence appears in the backend logs, not the terminal response. The request can fail from the user's perspective while the malicious code still executed successfully on the backend.

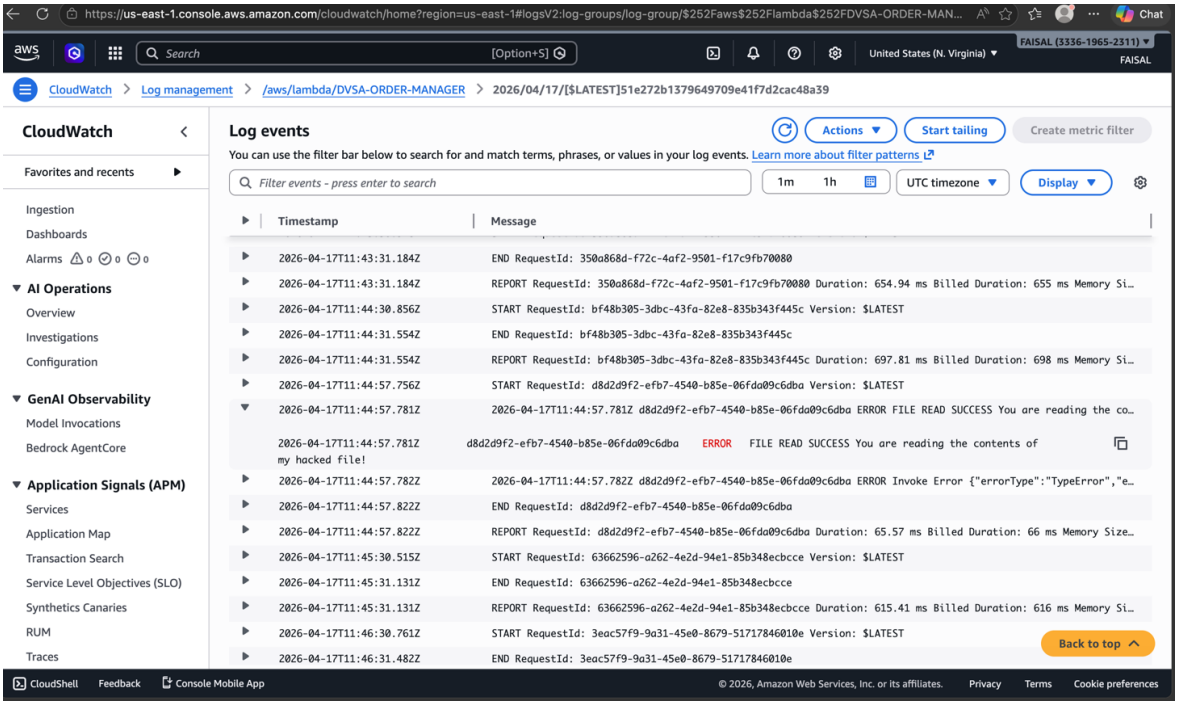
5. Evidence and Proof

The vulnerability is confirmed by two pieces of evidence:

- **Terminal response:** `{"message": "Internal server error"}` — this is normal API Gateway behavior masking the backend crash. It does not indicate the attack failed.
- **CloudWatch Logs** for `/aws/lambda/DVSA-ORDER-MANAGER` shows the injected log line:

```
FILE READ SUCCESS You are reading the contents of my hacked file!
```

This proves the injected JavaScript function executed on the Lambda backend at timestamp `~2026-04-17T11:44:57Z`. The injected function:



1. Wrote a file to `/tmp/pwned.txt` on the Lambda execution environment
2. Read it back
3. Logged the contents via `console.error()` — which appears in CloudWatch

The key lesson: the request failed at the API level but the malicious code still ran on the server.

6. Fix Strategy / Probable Mitigation

The fix is to **remove the node-serialize library entirely** and replace all unsafe `unserialize()` calls with `JSON.parse()`. Unlike `node-serialize`, `JSON.parse()` only parses plain data and cannot execute embedded functions.

Additionally:

- All incoming request fields should be validated against a strict allowlist before processing.
- Third-party packages with known CVEs should be removed or upgraded to safe alternatives.
- Input validation should occur before deserialization, not after.

7. Code / Config Changes

File changed: `order-manager.js` (DVSA-ORDER-MANAGER Lambda function)

Change: Replaced `require('node-serialize')` and `unserialize()` with `JSON.parse()`.

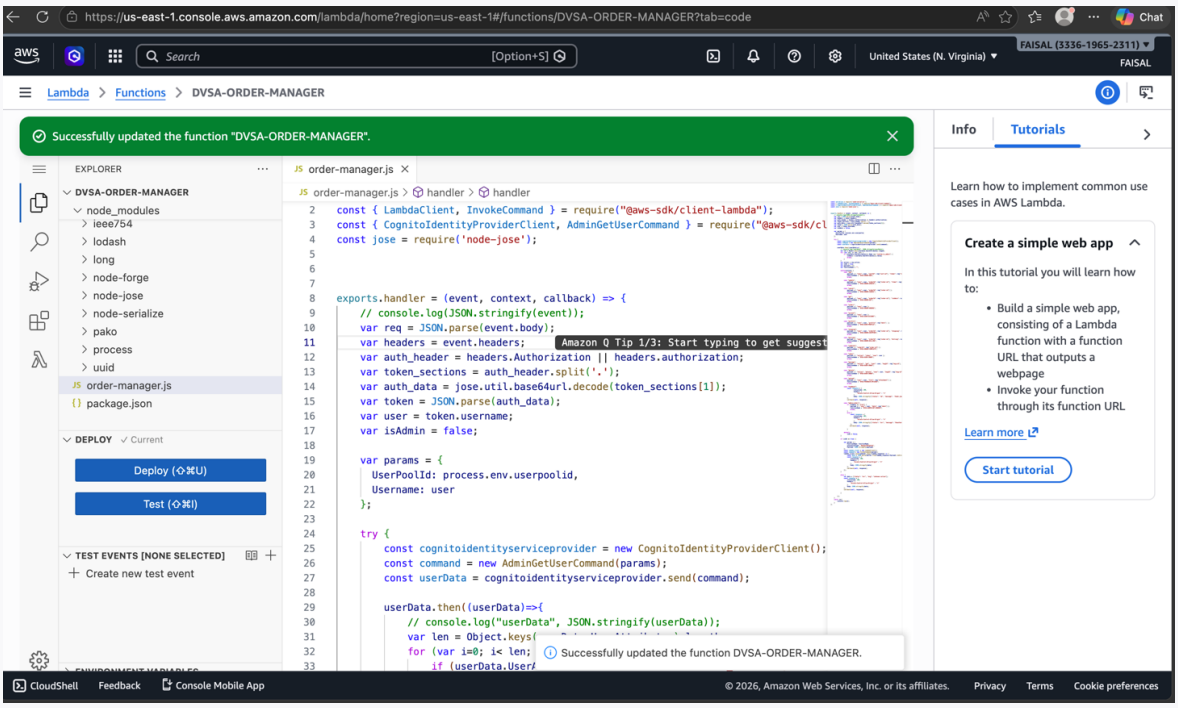
Vulnerable Code (Before)

```
// order-manager.js — VULNERABLE
var serialize = require('node-serialize');
var obj = serialize.unserialize(data);
```

No input validation is performed. The `unserialize()` call executes any embedded JavaScript function in the payload.

Fixed Code (After)

```
// order-manager.js — FIXED
var obj = JSON.parse(data);
```



`JSON.parse()` only handles plain JSON data. It cannot evaluate or execute function expressions, completely eliminating the code injection vector.

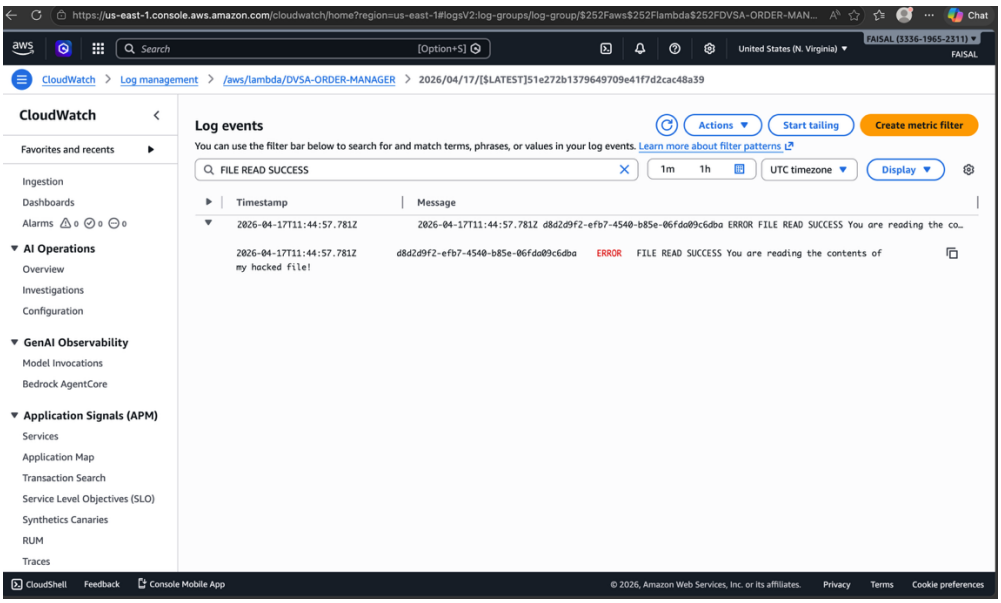
8. Verification After Fix

After deploying the fix, the same attack command was re-executed from the terminal.

The CloudWatch log group `/aws/lambda/DVSA-ORDER-MANAGER` was searched again for `FILE READ SUCCESS`.

The only matching entry found was from timestamp `2026-04-17T11:44:57Z` — logged **before** the fix was deployed.

No new `FILE READ SUCCESS` entries appeared after the fix, confirming the injected code no longer executes. The fix successfully blocked the event injection attack.



*After applying the fix, the same attack command was re-executed from the terminal. The CloudWatch log filter for `FILE READ SUCCESS` was searched again in the `/aws/lambda/DVSA-ORDER-MANAGER` log group. The only matching entry found was the one from timestamp `2026-04-17T11:44:57Z`, which was logged **before** the fix was deployed. No new `FILE READ SUCCESS` entries appeared after the fix deployment, confirming that the injected code was no longer executed by the Lambda function. The fix successfully blocked the event injection attack."*

9. Structured Operation and Security Analysis

Table A — Normal vs. Exploit Behavior

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Event Injection	The API must only process plain JSON data — no code execution is allowed from user input.	CloudWatch logs, `curl` terminal output, Lambda source code (`order-manager.js`)	Normal order requests return a valid JSON response with no backend code execution.	`FILE READ SUCCESS: You are reading the contents of my hacked file!` appeared in CloudWatch at ~11:44 after the injection payload was sent — confirming RCE.

Table B — Deviation and Fix Analysis

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Event Injection	The backend executed attacker-controlled JavaScript, violating the rule that user input must never be executed as code. The `node-serialize` library treated a crafted string as a callable function and invoked it during deserialization.	Intentional misuse / security-relevant abuse	Replaced `unserialize()` with `JSON.parse()` in `order-manager.js` (DVSA-ORDER-MANAGER Lambda). Removed `node-serialize` dependency entirely.	Re-ran the same attack after fix — no new `FILE READ SUCCESS` entry appeared in CloudWatch. The injection payload no longer executes.

10. Takeaway / Lessons Learned

1. **Never deserialize user input with a library that can execute code.** `node-serialize` was designed for a use case where the serializer and deserializer are both trusted parties. Using it on attacker-controlled input turns it into a remote code execution vector.

2. **A failed API response does not mean the attack failed.** The `{"message": "Internal server error"}` response masked a successful backend compromise. Defenders must check backend logs — not just API responses — to detect injection attacks.

3. **In serverless systems, RCE has immediate blast radius.** Lambda functions have file access, environment variables, and AWS service permissions. Code execution inside Lambda means the attacker can read credentials, write files, and invoke other AWS services.

4. **Dependency hygiene is mandatory.** The `node-serialize` package has a known CVE (CVE-2017-5941) published in 2017. Using it in a production Lambda function in 2026 represents a failure of dependency management. All third-party packages must be regularly audited with `npm audit` or equivalent.

5. **The fix is one line.** Replacing `unserialize(data)` with `JSON.parse(data)` completely eliminates the vulnerability. The cost of the fix is zero; the cost of the breach is full server-side code execution.